

SQL PRACTICE ASSIGNMENT

For Data Analyst Interns — Microsoft SQL Server (SSMS)

RetailAnalyticsDB | 105 Questions | Beginner → Intermediate → Hard

Topics covered: Views & Materialized (Indexed) Views, Analytical / Window Functions, Aggregate Functions, GROUP BY, HAVING, WHERE, Joins (Inner, Left, Right, Cross, Full Outer), Key Constraints, Indexing, Data Types, String & Date Functions, CREATE, INSERT, UPDATE, DELETE, DROP, TRUNCATE, CTE, COMMIT, ROLLBACK, SAVEPOINT, UNION, UNION ALL, MERGE.

Tool: Microsoft SQL Server Management Studio (SSMS)

How to use this document: Run the "Database Setup" script first to create the database, tables and sample data. Then attempt the questions in order — Beginner, Intermediate, and Hard. A separate Solutions PDF is provided, but try each question yourself first.

Part 1 — Database Setup Script

Run the script below in SSMS (connected to any SQL Server instance) to create the practice database `RetailAnalyticsDB` , its 8 tables, and load the sample data used throughout every question in this assignment. Run it top to bottom, once.

1.1 Create Database and Tables

```
IF DB_ID('RetailAnalyticsDB') IS NULL
    CREATE DATABASE RetailAnalyticsDB;
GO
USE RetailAnalyticsDB;
GO

CREATE TABLE Departments (
    DepartmentID INT IDENTITY(1,1) PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL UNIQUE,
    Location VARCHAR(50)
);

CREATE TABLE Employees (
    EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    HireDate DATE NOT NULL,
    JobTitle VARCHAR(50),
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    DepartmentID INT NULL REFERENCES Departments (DepartmentID),
    ManagerID INT NULL REFERENCES Employees (EmployeeID) -- self-referencing FK
);

CREATE TABLE Customers (
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    City VARCHAR(50),
    State VARCHAR(50),
    Country VARCHAR(50),
    JoinDate DATE DEFAULT GETDATE()
);

CREATE TABLE Categories (
    CategoryID INT IDENTITY(1,1) PRIMARY KEY,
    CategoryName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Suppliers (
    SupplierID INT IDENTITY(1,1) PRIMARY KEY,
    SupplierName VARCHAR(100) NOT NULL,
    City VARCHAR(50),
    Country VARCHAR(50),
    ContactEmail VARCHAR(100)
);

CREATE TABLE Products (
    ProductID INT IDENTITY(1,1) PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    CategoryID INT NULL REFERENCES Categories (CategoryID),
    SupplierID INT NULL REFERENCES Suppliers (SupplierID),
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice >= 0),
    StockQuantity INT DEFAULT 0
);

CREATE TABLE Orders (
    OrderID INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID INT NULL REFERENCES Customers (CustomerID),
    EmployeeID INT NULL REFERENCES Employees (EmployeeID),
    OrderDate DATE NOT NULL,
    ShipDate DATE NULL,
    Status VARCHAR(20) DEFAULT 'Pending'
);

CREATE TABLE OrderDetails (
    OrderDetailID INT IDENTITY(1,1) PRIMARY KEY,
    OrderID INT NOT NULL REFERENCES Orders (OrderID),
    ProductID INT NOT NULL REFERENCES Products (ProductID),
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) NOT NULL,
    Discount DECIMAL(4,2) DEFAULT 0
);
GO
```

1.2 Load Sample Data

```
INSERT INTO Departments (DepartmentName, Location) VALUES
('Sales', 'Kolkata'), ('Marketing', 'Mumbai'), ('IT', 'Hyderabad'), ('HR', 'Bengaluru'), ('Finance', 'Delhi');

INSERT INTO Employees (FirstName, LastName, Email, Phone, HireDate, JobTitle, Salary, DepartmentID, ManagerID) VALUES
('Arjun', 'Sharma', 'arjun.sharma@retailcorp.com', '9831122334', '2018-03-15', 'Sales Manager', 65000, 1, NULL),
('Priya', 'Das', 'priya.das@retailcorp.com', '9831122335', '2019-06-01', 'Sales Executive', 42000, 1, 1),
('Rohit', 'Verma', 'rohit.verma@retailcorp.com', '9831122336', '2020-01-10', 'Sales Executive', 40000, 1, 1),
('Sneha', 'Roy', 'sneha.roy@retailcorp.com', '9831122337', '2017-11-20', 'Marketing Manager', 70000, 2, NULL),
('Kabir', 'Khan', 'kabir.khan@retailcorp.com', '9831122338', '2021-02-14', 'Marketing Executive', 38000, 2, 4),
```

```
('Ananya','Gupta','ananya.gupta@retailcorp.com','9831122339','2019-09-05','IT Manager',85000,3,NULL),
('Vikram','Singh','vikram.singh@retailcorp.com','9831122340','2022-04-18','Data Analyst',45000,3,6),
('Ishita','Mukherjee','ishita.mukherjee@retailcorp.com','9831122341','2023-01-09','Data Analyst Intern',20000,3,6),
('Rajesh','Nair','rajesh.nai@retailcorp.com','9831122342','2018-07-22','HR Manager',60000,4,NULL),
('Meera','Iyer','meera.iyer@retailcorp.com','9831122343','2020-10-30','HR Executive',35000,4,9),
('Suresh','Menon','suresh.menon@retailcorp.com','9831122344','2017-05-12','Finance Manager',72000,5,NULL),
('Divya','Pillai','divya.pillai@retailcorp.com','9831122345','2021-08-25','Finance Executive',41000,5,11);
```

```
INSERT INTO Customers (FirstName,LastName,Email,City,State,Country,JoinDate) VALUES
('Aditya','Bose','aditya.bose@gmail.com','Kolkata','West Bengal','India','2022-01-15'),
('Riya','Chatterjee','riya.chatterjee@gmail.com','Kolkata','West Bengal','India','2022-02-20'),
('Karan','Malhotra','karan.malhotra@gmail.com','Delhi','Delhi','India','2021-11-05'),
('Neha','Kapoor','neha.kapoor@gmail.com','Mumbai','Maharashtra','India','2023-03-10'),
('Arnav','Joshi','arnav.joshi@gmail.com','Pune','Maharashtra','India','2022-07-19'),
('Simran','Kaur','simran.kaur@gmail.com','Chandigarh','Punjab','India','2023-05-01'),
('Farhan','Ali','farhan.ali@gmail.com','Hyderabad','Telangana','India','2021-09-12'),
('Pooja','Reddy','pooja.reddy@gmail.com','Hyderabad','Telangana','India','2022-12-03'),
('Devansh','Trivedi','devansh.trivedi@gmail.com','Ahmedabad','Gujarat','India','2023-06-25'),
('Kavya','Nambiar','kavya.nambiar@gmail.com','Bengaluru','Karnataka','India','2021-04-08'),
('Rohan','Bhatt','rohan.bhatt@gmail.com','Bengaluru','Karnataka','India','2022-10-17'),
('Anjali','Desai','anjali.desai@gmail.com','Surat','Gujarat','India','2023-02-14'),
('James','Carter','james.carter@gmail.com','New York','NY','USA','2022-08-09'),
('Emily','Clark','emily.clark@gmail.com','London','England','UK','2023-01-22'),
('Wei','Zhang','wei.zhang@gmail.com','Singapore','Singapore','Singapore','2022-05-30');
```

```
INSERT INTO Categories (CategoryName) VALUES
('Electronics'),('Grocery'),('Apparel'),('Furniture'),('Stationery'),('Beauty');
```

```
INSERT INTO Suppliers (SupplierName,City,Country,ContactEmail) VALUES
('TechSource Pvt Ltd','Bengaluru','India','contact@techsource.com'),
('FreshFarm Distributors','Kolkata','India','sales@freshfarm.com'),
('UrbanThreads Apparel','Mumbai','India','info@urbanthreads.com'),
('HomeStyle Furniture Co','Chennai','India','support@homestyle.com'),
('WriteWell Stationers','Delhi','India','hello@writewell.com'),
('GlowBeauty Imports','Hyderabad','India','sales@glowbeauty.com');
```

```
INSERT INTO Products (ProductName,CategoryID,SupplierID,UnitPrice,StockQuantity) VALUES
('Wireless Mouse',1,1,599,150),('Bluetooth Headphones',1,1,1999,80),
('USB-C Charger',1,1,799,200),('Laptop Stand',1,1,1299,60),
('Basmati Rice 5kg',2,2,650,300),('Organic Honey 500g',2,2,320,120),
('Green Tea Pack',2,2,180,250),('Almonds 1kg',2,2,900,90),
('Men's Cotton T-Shirt',3,3,499,200),('Women's Kurti',3,3,899,150),
('Denim Jeans',3,3,1499,100),('Winter Jacket',3,3,2999,40),
('Office Chair',4,4,4999,25),('Study Table',4,4,3499,30),
('Bookshelf',4,4,2799,20),('Notebook Pack of 5',5,5,150,400),
('Gel Pen Set',5,5,99,500),('Sketchbook',5,5,199,220),
('Face Moisturizer',6,6,349,130),('Herbal Shampoo',6,6,279,160);
```

```
INSERT INTO Orders (CustomerID,EmployeeID,OrderDate,ShipDate,Status) VALUES
(1,2,'2023-01-05','2023-01-08','Delivered'),(2,3,'2023-01-12','2023-01-15','Delivered'),
(3,2,'2023-01-20','2023-01-25','Delivered'),(4,1,'2023-02-02','2023-02-06','Delivered'),
(5,3,'2023-02-14',NULL,'Cancelled'),(1,2,'2023-02-28','2023-03-02','Delivered'),
(6,2,'2023-03-10','2023-03-14','Delivered'),(7,3,'2023-03-15','2023-03-20','Delivered'),
(8,1,'2023-03-22',NULL,'Pending'),(2,2,'2023-04-01','2023-04-05','Delivered'),
(9,3,'2023-04-11','2023-04-15','Delivered'),(10,2,'2023-04-18','2023-04-22','Delivered'),
(3,1,'2023-05-02','2023-05-06','Delivered'),(11,3,'2023-05-15',NULL,'Cancelled'),
(12,2,'2023-05-25','2023-05-29','Delivered'),(4,1,'2023-06-05','2023-06-10','Delivered'),
(13,2,'2023-06-18','2023-06-24','Delivered'),(14,3,'2023-07-01','2023-07-07','Delivered'),
(15,1,'2023-07-15',NULL,'Pending'),(5,2,'2023-08-02','2023-08-07','Delivered'),
(1,3,'2023-09-10','2023-09-14','Delivered'),(7,1,'2023-10-05','2023-10-10','Delivered'),
(9,2,'2023-11-20','2023-11-25','Delivered'),(2,3,'2024-01-08','2024-01-12','Delivered'),
(6,1,'2024-02-14',NULL,'Pending');
```

```
INSERT INTO OrderDetails (OrderID,ProductID,Quantity,UnitPrice,Discount) VALUES
(1,1,2,599,0),(1,3,1,799,0),(2,5,3,650,0.05),(2,6,2,320,0),
(3,9,2,499,0),(3,11,1,1499,0.10),(4,13,1,4999,0),(4,16,5,150,0),
(5,2,1,1999,0),(6,1,1,599,0),(6,4,1,1299,0),(7,19,2,349,0),
(7,20,2,279,0),(8,10,1,899,0),(8,12,1,2999,0.15),(9,17,10,99,0),
(9,18,5,199,0),(10,5,2,650,0),(10,8,1,900,0),(11,2,2,1999,0.05),
(12,14,1,3499,0),(12,15,1,2799,0),(13,1,3,599,0),(13,3,2,799,0),
(14,9,4,499,0),(15,6,1,320,0),(15,7,2,180,0),(16,13,2,4999,0.10),
(17,11,1,1499,0),(17,12,1,2999,0),(18,16,3,150,0),(18,17,5,99,0),
(19,19,1,349,0),(20,2,1,1999,0),(20,4,1,1299,0),(21,5,4,650,0),
(21,8,2,900,0),(22,9,3,499,0),(22,10,2,899,0),(23,1,5,599,0.05),
(24,20,3,279,0),(24,19,3,349,0),(25,6,2,320,0),(25,7,4,180,0);
GO
```

All 105 questions below assume this exact schema and data. Table/column names, IDs and sample values referenced in the questions correspond directly to what you just created. Run each question's answer in SSMS against [RetailAnalyticsDB](#).

1.3 Entity Reference (for quick lookup while solving)

Table	Key Columns	Notes
Departments	DepartmentID (PK)	5 rows: Sales, Marketing, IT, HR, Finance
Employees	EmployeeID (PK), DepartmentID (FK), ManagerID (FK → self)	12 rows across 5 departments
Customers	CustomerID (PK)	15 rows, mostly India + 3 international
Categories	CategoryID (PK)	6 rows
Suppliers	SupplierID (PK)	6 rows
Products	ProductID (PK), CategoryID (FK), SupplierID (FK)	20 rows
Orders	OrderID (PK), CustomerID (FK), EmployeeID (FK)	25 rows, statuses: Delivered / Pending / Cancelled
OrderDetails	OrderDetailID (PK), OrderID (FK), ProductID (FK)	44 rows (line items)

Part 2 — Practice Questions

LEVEL 1 — BEGINNER (Q1 - Q52)

Focus: fundamentals of every topic — get comfortable with syntax before moving to Level 2.

Data Types & Table Creation

- Q1** Which SQL Server data type would you use for `OrderDate` in the `Orders` table, and why not `DATETIME` here?
- Q2** Write a query using `INFORMATION_SCHEMA.COLUMNS` (or `sys.columns`) to check the data type and length of the `Salary` column in `Employees`.
- Q3** Create a new table `Reviews` with: `ReviewID` (INT, PK, identity), `ProductID` (INT), `Rating` (INT, must be between 1 and 5), `ReviewText` (VARCHAR(500)), `ReviewDate` (DATE, defaults to today).
- Q4** Explain, in your own words, the difference between `CHAR`, `VARCHAR` and `TEXT` data types in SQL Server, and when you would pick each.

Key Constraints

- Q5** Identify the Primary Key and every Foreign Key in the `Orders` table. Explain why `ManagerID` in `Employees` is called a "self-referencing" foreign key.
- Q6** Write an `ALTER TABLE` statement to add a `UNIQUE` constraint on the `Phone` column of `Employees`.

CREATE

- Q7** Write a `CREATE TABLE` statement for `Wishlist` with columns `CustomerID`, `ProductID`, `DateAdded`, using a **composite primary key** of (`CustomerID`, `ProductID`).
- Q8** Create a table `Coupons` with `CouponCode` VARCHAR(10) as PK, `DiscountPercent` DECIMAL(4,2), `ExpiryDate` DATE, and `IsActive` BIT defaulting to 1.

INSERT

- Q9** Insert a new department `'Customer Support'` located in `'Pune'` into `Departments`.
- Q10** Insert a new customer `'Tanya Bhatt'` from `'Jaipur'`, `'Rajasthan'`, `'India'` with email `tanya.bhatt@gmail.com`.
- Q11** Insert three new products of your choice into the `Stationery` category using a **single** INSERT statement with multiple `VALUES` rows.

WHERE Clause

- Q12** Retrieve all products with `UnitPrice` greater than 1000.
- Q13** Retrieve all customers whose `City` is `'Kolkata'` or `'Hyderabad'`.
- Q14** Retrieve all orders placed between `'2023-03-01'` and `'2023-06-30'` (inclusive).
- Q15** Retrieve all employees whose `JobTitle` contains the word `'Manager'`.
- Q16** Retrieve all orders where `ShipDate` is `NULL`.

UPDATE

- Q17** Increase the `Salary` of every employee in the IT department by 10%.
- Q18** Update Order `9`'s `Status` to `'Shipped'` and set its `ShipDate` to `'2023-03-26'`.
- Q19** Reduce the `StockQuantity` of `'Wireless Mouse'` by 10 (simulate a manual stock correction).

DELETE

- Q20** Delete all rows from `OrderDetails` that belong to an order whose `Status` = `'Cancelled'`.
- Q21** Write a **safe** DELETE (using `NOT EXISTS`) that removes `'Wei Zhang'` from `Customers` only if they have never placed an order.

Aggregate Functions

- Q22** Find the total number of products in the `Products` table.
- Q23** Find the average `UnitPrice` across all products, rounded to 2 decimals.
- Q24** Find the maximum and minimum `Salary` among all employees in a single query.
- Q25** Find the total quantity of items sold across all order details (`SUM(Quantity)`).
- Q26** Find how many **distinct** customers have placed at least one order.

GROUP BY

- Q27** Find the number of employees in each department.
- Q28** Find the sum of `UnitPrice` of products, grouped by `CategoryID`.
- Q29** Find the total quantity sold for each `ProductID` in `OrderDetails`.
- Q30** Find the number of orders placed by each customer.

HAVING

- Q31** Find departments that have **more than 2** employees.
- Q32** Find products whose total ordered quantity (summed across all orders) is **greater than 5**.
- Q33** Find customers who have placed **more than 1** order.

Joins — INNER JOIN

- Q34** List all orders along with the customer's first and last name.
- Q35** List all products along with their category name.
- Q36** List all order-detail line items along with the product name and the order date.
- Q37** List all employees with the name of the department they belong to.

Joins — LEFT JOIN

- Q38** List every customer and their orders, **including** customers who have never placed an order.
- Q39** List every employee with their department name, **including** any employee not yet assigned to a department.
- Q40** List every product with its total quantity ordered, **including** products that have never been ordered (show 0 or NULL).

Joins — RIGHT JOIN

- Q41** Using `RIGHT JOIN`, list all departments and their employees, including departments that currently have zero employees.
- Q42** Using `RIGHT JOIN`, list all categories and their products, including categories with no products.

Joins — CROSS JOIN

- Q43** Generate every possible combination of `Department` × `Category` (a Cartesian product) — useful for building a blank coverage/report matrix.
- Q44** Use `CROSS JOIN` to pair every employee in the **IT** department with every product in the **Electronics** category.

Joins — FULL OUTER JOIN

- Q45** Use `FULL OUTER JOIN` to combine `Customers` and `Orders` so that customers without orders (and any orphaned orders, if they existed) both appear.
- Q46** Use `FULL OUTER JOIN` to combine `Employees` and `Orders` on `EmployeeID`, so employees who have never handled an order still appear in the result.

String Functions

- Q47** Display each customer's full name as `'LastName, FirstName'` using `CONCAT()`.
- Q48** Display all product names in **uppercase** using `UPPER()`.
- Q49** Find the length of each customer's email address using `LEN()`.
- Q50** Extract the first 3 characters of every `ProductName` using `LEFT()`.

Date Functions

- Q51** Extract just the **year** from `OrderDate` for every order using `YEAR()`.
- Q52** Calculate the number of days between `OrderDate` and `ShipDate` for each order using `DATEDIFF()`.

LEVEL 2 — INTERMEDIATE (Q53 - Q80)

Focus: combining concepts — views, CTEs, window functions, transactions, indexing.

Views

- Q53** Create a view `vw_CustomerOrderSummary` showing `CustomerID`, full customer name, `TotalOrders`, and `TotalAmountSpent` (join `Customers`, `Orders`, `OrderDetails`, grouped by customer).
- Q54** Create a view `vw_ProductSales` showing `ProductID`, `ProductName`, `CategoryName`, `TotalQuantitySold`, and `TotalRevenue`.
- Q55** Write a query against `vw_ProductSales` to retrieve the top 5 products ordered by `TotalRevenue` descending.

CTE (Common Table Expressions)

- Q56** Write a CTE `DeptEmpCount` that calculates employee count per department, then select only departments with count > 2 from the CTE.
- Q57** Write a CTE that calculates each customer's total spend, then join it back to `Customers` to show the customer's name and spend for those who spent more than 5000.
- Q58** Write a query using **two chained CTEs**: the first computes monthly order revenue totals, the second ranks the months by revenue — then select the top 3 months.
- Q59** Use a CTE with a window function to identify the **2nd highest-priced** product in each category.

Analytical / Window Functions

- Q60** Use `ROW_NUMBER()` to rank employees within each department by `Salary` descending.
- Q61** Use `RANK()` and `DENSE_RANK()` together to rank products within each category by `UnitPrice`, and explain the difference you observe.
- Q62** Use `LAG()` to show, for each customer's orders (ordered by date), the `OrderDate` of that customer's **previous** order alongside the current one.
- Q63** Use `LEAD()` to show, for employees ordered by `HireDate`, the `HireDate` of the **next** employee hired.
- Q64** Use `NTILE(4)` to split all products into 4 price-based buckets (quartiles).

UNION / UNION ALL

- Q65** Use `UNION` to produce one combined, de-duplicated list of all `City` values from `Customers` and `Suppliers`.
- Q66** Use `UNION ALL` to combine a list of "High Value" customers (spent > 5000) with a list of "Frequent" customers (more than 2 orders), keeping a customer twice if they qualify for both.
- Q67** Using an example from this schema, explain why `UNION ALL` is generally faster than `UNION`.

MERGE

- Q68** Create a temp table `#ProductPriceUpdates(ProductID, NewPrice)` with a couple of sample rows, then write a `MERGE` statement that updates `Products.UnitPrice` for any matching `ProductID`.
- Q69** Create a table `OrderAudit(AuditID INT IDENTITY PK, ProductID INT, ChangeType VARCHAR(20), ChangeDate DATETIME DEFAULT GETDATE())`. Write a `MERGE` that, while syncing a stock-count staging table into `Products`, inserts an audit row via the `OUTPUT` clause whenever a row is `WHEN MATCHED VS WHEN NOT MATCHED`.

Indexing

- Q70** Create a non-clustered index on `Orders.CustomerID` and explain why it can speed up the join queries you wrote earlier.
- Q71** Create a unique index on `Customers.Email`.
- Q72** Explain the difference between a **clustered** and a **non-clustered** index, referencing the `Employees` table's primary key as your example.

Transactions — COMMIT / ROLLBACK / SAVEPOINT

- Q73** Write a transaction that inserts a new order and its order-detail rows; if an error is raised, `ROLLBACK`, otherwise `COMMIT` (use `TRY...CATCH`).
- Q74** Demonstrate SQL Server's savepoint syntax (`SAVE TRANSACTION`): begin a transaction, insert a new customer, create a savepoint, insert an order for that customer, then roll back **only to the savepoint** (undoing just the order, keeping the customer), then `COMMIT`.
- Q75** Explain what happens to locks and the transaction log when you `COMMIT` normally, versus when you issue a full `ROLLBACK` after an error partway through a multi-statement transaction.

Materialized View (Indexed View in SQL Server)

- Q76** SQL Server implements materialized views as **indexed views**. Create a view `vw_CategoryRevenue` with `SCHEMABINDING` that stores total revenue per category, then create a **unique clustered index** on it to materialize it.
- Q77** Explain two restrictions SQL Server places on a view before it is allowed to be indexed (materialized).

DROP / TRUNCATE

Q78 Write a statement to TRUNCATE the Reviews table you created earlier, and explain why TRUNCATE is generally faster than a DELETE with no WHERE clause.

Q79 Explain the difference between DROP TABLE and TRUNCATE TABLE, including what happens to the identity seed in each case.

Joins + Aggregation (combined)

Q80 Join Orders, OrderDetails and Products to find the total revenue generated by each employee (through the orders they handled), sorted highest to lowest.

LEVEL 3 — HARD / LOGICAL (Q81 - Q105)

Focus: interview-style reasoning — window functions, recursive CTEs, performance thinking, MERGE with all clauses.

- Q81** Find the **second highest** `UnitPrice` product overall **without** using `TOP` or `OFFSET-FETCH` (use a subquery or window function instead).
- Q82** For each department, find the employee with the **highest salary**, correctly handling possible ties, using a window function.
- Q83** Find customers who have ordered a product from **every single category** that exists in the `Categories` table (a "relational division" style query).
- Q84** Find the **top 3 products by revenue** within each category using `ROW_NUMBER() ... PARTITION BY CategoryID`.
- Q85** Calculate a **running (cumulative) total** of order revenue over time using a window function.
- Q86** Calculate a **3-order moving average** of order revenue per customer using a window function with a frame clause.
- Q87** Using a **self-join**, find pairs of employees who share the same `JobTitle`, excluding an employee being paired with themselves and excluding duplicate mirrored pairs (A,B / B,A).
- Q88** Identify any "gaps" (missing values) in the `OrderID` sequence — useful for checking whether orders were ever deleted.
- Q89** Explain and write the general "gaps-and-islands" logic to identify runs of **consecutive days** on which a given customer placed orders.
- Q90** Find customers whose total spend is **above the average** total spend across all customers, using a subquery.
- Q91** Find the calendar month with the **highest total revenue** using a CTE combined with `RANK()`.
- Q92** Write a **recursive CTE** that displays the full management chain (employee → manager → manager's manager → ...) for a given employee, e.g. `EmployeeID = 2`.
- Q93** For each customer, find the number of days between their **first** order and their **most recent** order.
- Q94** Find products that have **never been ordered**, using a `LEFT JOIN ... IS NULL` pattern (do **not** use `NOT IN`).
- Q95** Find the employee who generated the **highest total revenue**, and what percentage of total company revenue that represents.
- Q96** Create a staging table `#StockCount(ProductID, CountedQty)`. Write a single `MERGE` statement that: updates `Products.StockQuantity` where the counted quantity differs (`WHEN MATCHED`), inserts any brand-new products present only in the staging table (`WHEN NOT MATCHED BY TARGET`), and deletes products that are discontinued — present in `Products` but absent from the feed (`WHEN NOT MATCHED BY SOURCE`).
- Q97** Using `UNION ALL` and `CASE`, build one result set combining "Sale" and "Cancellation" transactions (based on `Orders.Status`) with a `TransactionType` column, then aggregate **net** revenue per product.
- Q98** Write a short transaction demonstrating a scenario where **skipping a SAVEPOINT** forces you to roll back more work than necessary. Then rewrite it **with** a savepoint to avoid the problem.
- Q99** You are building a reporting dashboard that filters `Orders` by an `OrderDate` range and groups results by `CategoryName`. Decide which columns you would index, justify each choice in a comment, and write the corresponding `CREATE INDEX` statements.
- Q100** Write a query to detect potential duplicate customer records based on `Email`, comparing **case-insensitively**.
- Q101** Find the top-spending customer **within each State** using `RANK() ... PARTITION BY State`.
- Q102** For each employee, show their `Salary` alongside the difference between their salary and their **department's average salary**, using a window function.
- Q103** Using `HAVING` together with a CTE, find categories whose **average product price** is higher than the overall average product price across all categories.
- Q104** Using a CTE combined with a window function, flag every order as `'First Order'` or `'Repeat Order'` depending on whether it is that customer's earliest order.
- Q105** A junior analyst wrote `SELECT * FROM Orders WHERE YEAR(OrderDate) = 2023` and it runs slowly even though `OrderDate` is indexed. Explain **why** (non-sargable predicate) and rewrite the query so the index can actually be used.

End of question set — 105 questions total (52 Beginner · 28 Intermediate · 25 Hard). See the companion Solutions PDF for fully worked T-SQL answers.